

Individual Project 1(ED319)

Round Robin Arbiter Implementation in Verilog HDL

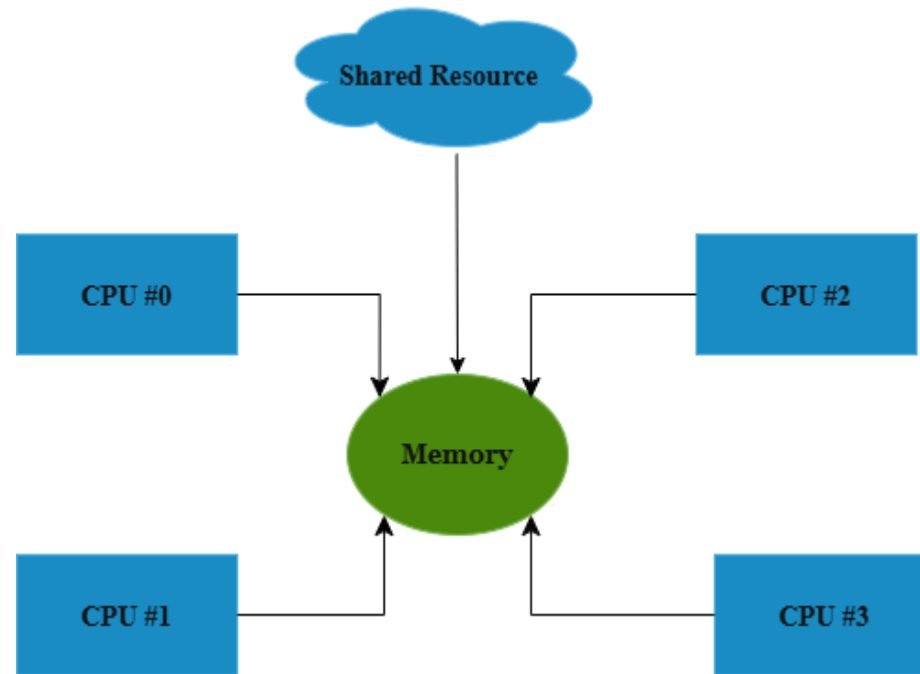
Name: Jaitra J. Solanki(202304005)

Project Mentor: Sreeja Rajendran

Problem Statement & Need for an Arbiter

Problem in Digital Systems

- Multiple modules request access to a **single shared resource** at the same time.
- Without control, this leads to **conflicts** and **unpredictable system behaviour**.



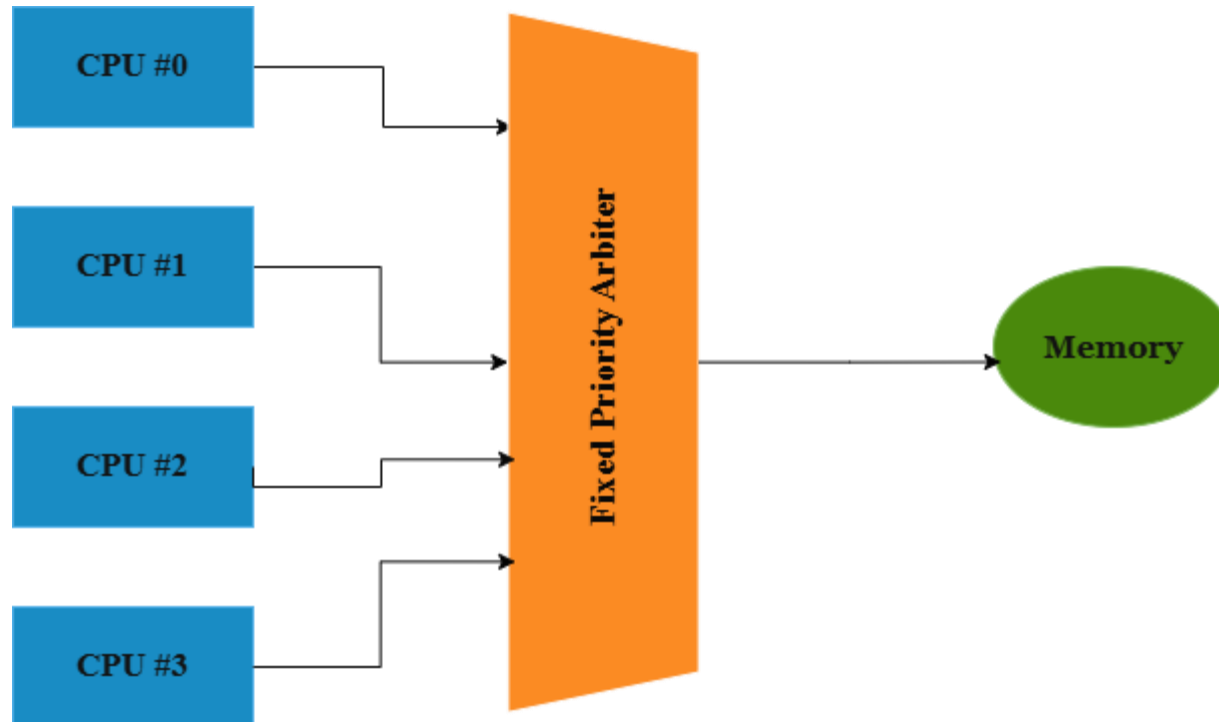
Arbiter

- An arbiter is a digital hardware block that decides which requestor gets access to a shared resource when multiple requestors ask for it at the same time.
- Common Arbitration Techniques:
 - 1) Fixed Priority Arbiter
 - 2) Round Robin Arbiter
 - 3) Matrix Arbiter

Now, **why do We Need an Arbiter**

- To decide which requester gets access at any given time.
- To prevent conflicts and ensure reliable communication.
- To provide an organised and controlled way of sharing resources.

Basic Block of Fixed-Priority Arbiter:



Application/Examples:

- To control bus access, memory access, or peripheral access.
- Microprocessors / Microcontrollers – interrupt handling.
- Shared Bus Systems – deciding which master gets the bus.
- In a smartphone SoC, both the CPU and GPU may request access to the same memory. The arbiter decides which one gets access first, preventing data collision and ensuring smooth operation.

Fixed-Priority Arbiter:

Advantage:

- Allows preference to a specific requester based on predefined priority levels.
- Useful in systems where certain tasks must always be serviced first. (In many microcontrollers, some interrupts must be serviced immediately (like timer interrupt or emergency fault interrupt).)

Disadvantage:

- When a high-priority requester keeps requesting frequently for a resource, it will result in starvation of requesters with low priority. (A process is ready to run, but it never gets CPU time because other processes keep getting started.)

A solution to starvation-free Arbitration is the **Round Robin Arbiter**.

Fixed Priority Arbiter – Verilog Simulation

Explanation of REQ[3:0] and GNT[3:0]:

- What is REQ[3:0]?
 - REQ[3:0] is a 4-bit input request bus.
Each bit represents a separate requester:
 - REQ[3] → Requester 3 (highest priority)
 - REQ[2] → Requester 2
 - REQ[1] → Requester 1
 - REQ[0] → Requester 0 (lowest priority)
- What does the 4-bit value represent?
 - Example: 1010
 - REQ[3] = 1 → Requester 3 is asking
 - REQ[2] = 0 → Requester 2 is not asking
 - REQ[1] = 1 → Requester 1 is asking
 - REQ[0] = 0 → Requester 0 is not asking

So this 4-bit number tells **which requesters are active at the same time.**

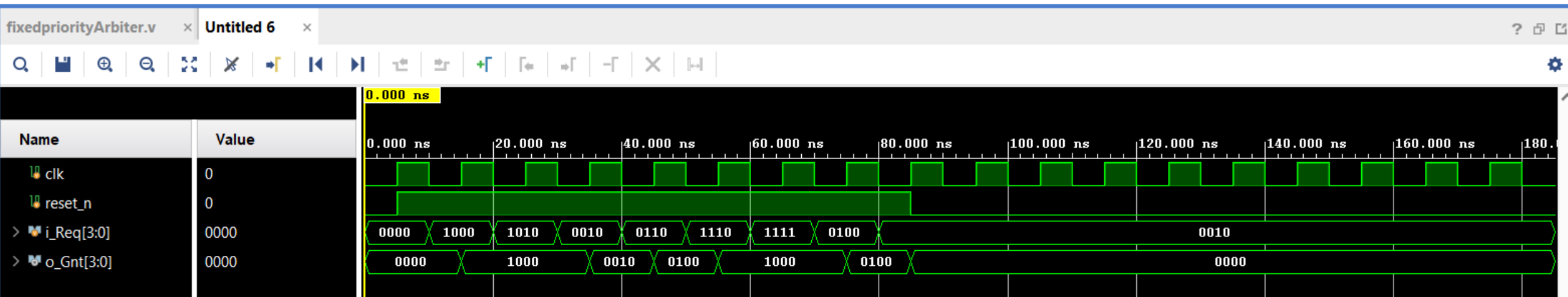


Fixed Priority Arbiter – Verilog Simulation

- **What is GNT[3:0]?**
 - GNT[3:0] is a 4-bit one-hot output grant bus.
 - Only one bit becomes 1 at a time — the requester who receives the grant:
 - GNT[3] = 1 → Requester 3 is granted
 - GNT[2] = 1 → Requester 2 is granted
 - GNT[1] = 1 → Requester 1 is granted
 - GNT[0] = 1 → Requester 0 is granted
- **From Our Waveform:**
 - When REQ=1010 the highest-priority requester active is REQ[3].
 - So GNT=1000 (grant goes to requester 3).
 - When REQ=0110, the highest active is REQ[2].
 - So, GNT=0100.

Fixed Priority Arbiter – Verilog Simulation

- The arbiter takes 4 input requests and generates a one-hot grant output based on fixed priority. The priority order is $REQ[3] > REQ[2] > REQ[1] > REQ[0]$, so the highest active request is always granted first.



Round Robin Arbiter

- A Round Robin Arbiter provides fair access to all requesters by rotating priorities after each grant. With a fixed time slice, each requester can use the resource for a set number of clock cycles before the turn moves to the next requester.
- **Advantage:** Shared Resource is allocated uniformly to all requesters and hence does not lead to the requester's starvation.
- **Disadvantage:** Since all requestors are treated equally, we can not give importance to a specific requester.
- **Basic Terminology:**
 - 1)**Burst Time:** The total amount of time a process needs to run on the CPU to finish its execution.
 - 2) **Quantum Time (Time Slice):** The fixed amount of CPU time given to each process before switching to the next one. Used in Round Robin scheduling.

Round Robin Arbiter(Fixed Time slices)

➤ How is Burst Time decided?

- Burst time depends on how much work a process has to do.
- If a process needs more CPU work, its burst time is longer.
- If the work is small, the burst time is short.
- So burst time is different for every process and comes from the task itself.

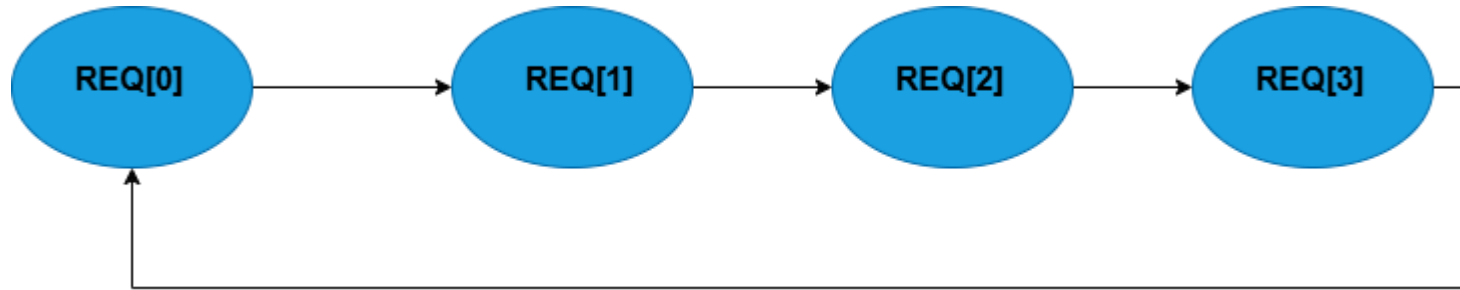
➤ How is Quantum Time decided?

- Quantum time is chosen by the system designer.
- It should not be too small, otherwise the CPU keeps switching between processes too often.
- It should not be too big, otherwise one process will take too long, and others will have to wait. So the designer picks a middle value.
- Not too small, not too big
- Just enough so that every process gets a fair chance and the system runs smoothly.

➤ Typical Quantum Time Ranges:

1. Operating Systems (Windows, Linux) → 4ms to 10ms
2. Smartphones (Android) → 1 ms to 5 ms
3. Real-Time Embedded Systems → (100 μ s – 1 ms)

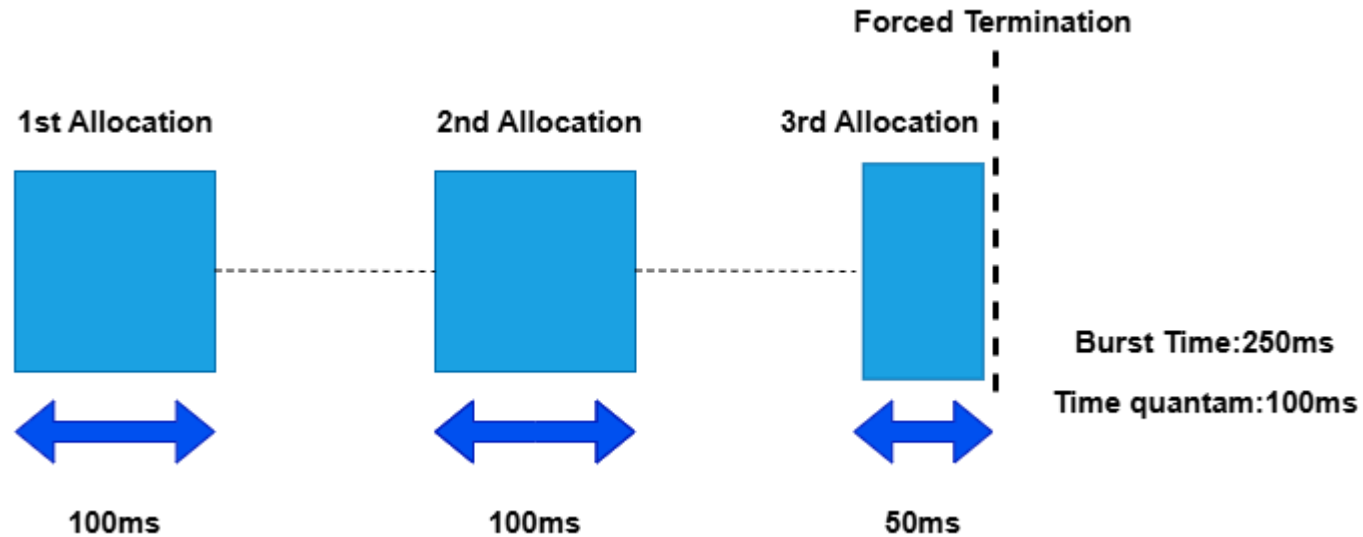
Basic Algorithm for Round Robin Arbiter



Example (Round Robin with Fixed Time Slice)

- Time Slice (Quantum Time) = 100 ms
- Process 1 Burst Time = 250 ms
- Process 1 runs for 100 ms, then the arbiter/CPU switches to the next process.
- After other processes get their turn, Process 1 continues again until its total is 250 ms completed.

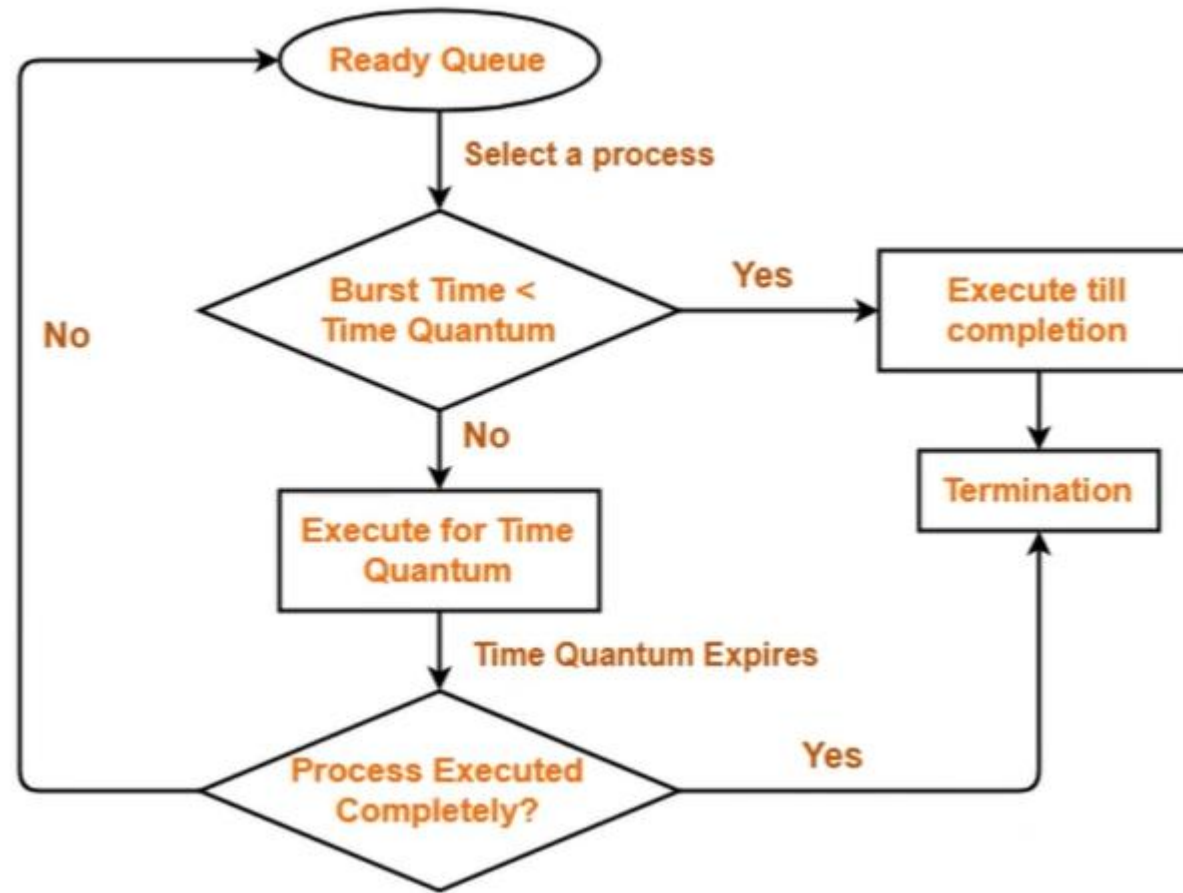
Basic Algorithm for Round Robin Arbiter



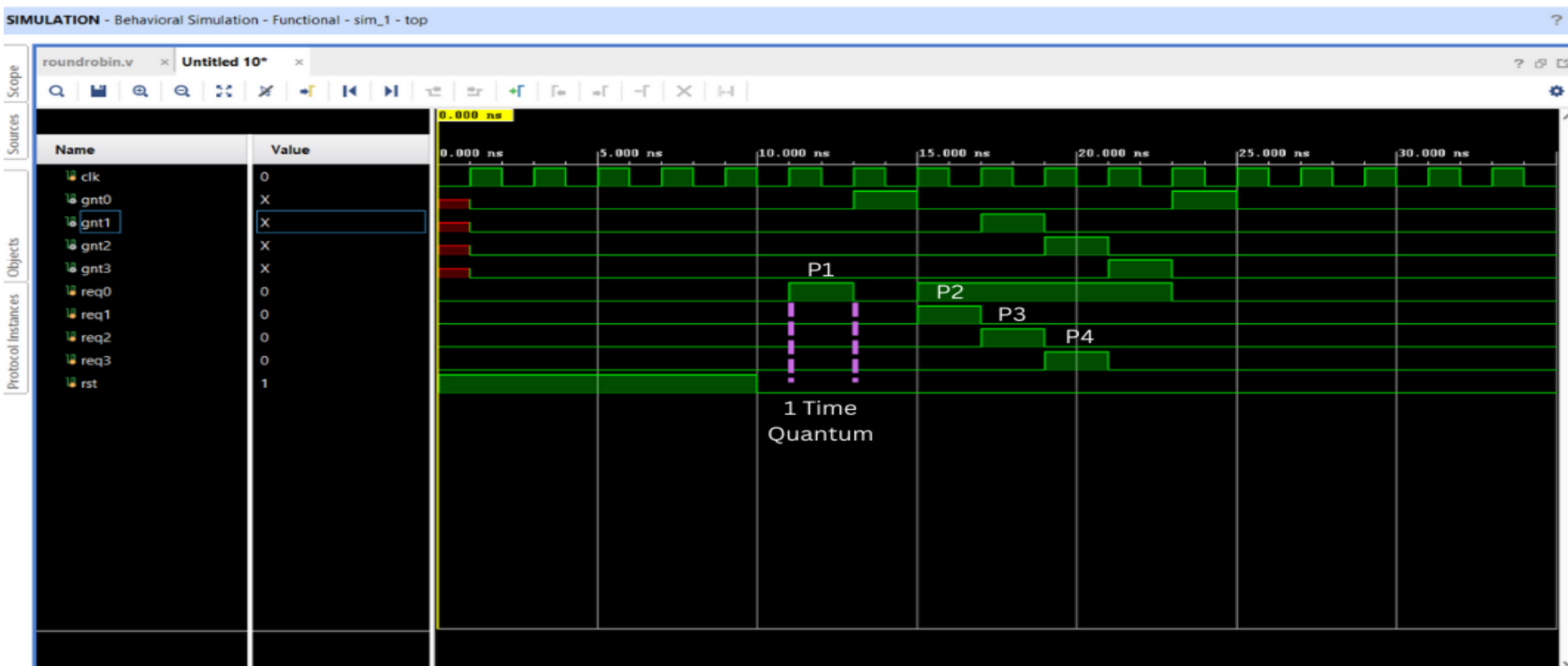
➤ Working of Round Robin Arbiter:

- The arbiter checks all requesters in a cyclic order (REQ0 → REQ1 → REQ2 → REQ3 → REQ0 → ...).
- A pointer keeps track of whose turn it is to get the next chance.
- When a requester gets the grant, the pointer moves to the next requester.
- If the current requester is not requesting, the arbiter moves to the next one in the cycle.
- This rotation ensures fairness and no starvation.
- With a fixed time slice, each requester holds the grant only for that time, then the arbiter switches.

Round Robin Arbiter – Scheduling Flow Chart



Round Robin Arbiter – Verilog Simulation



Round Robin Arbiter(Fixed Time Slices)

➤ Disadvantage:

1. Short tasks finish late

Short/quick tasks may need only a few cycles,
but RR gives them a full-time slice and also makes them wait behind all other tasks.

Example: Task A needs only 2 cycles

Tasks B, C, and D are also requesting

→ A gets only 1 time slice per round

→ A must wait for B, C, and D to finish their slices before it runs again

→ Unnecessary waiting → higher completion time

So even though A is short, it finishes slowly.

2. Long tasks keep getting interrupted

Long tasks need continuous execution,

but Round Robin stops them after every time slice and gives the turn to the next requester.

This leads to: more context switching, losing momentum, taking more total time to finish

So long tasks complete more slowly than if they were allowed to run continuously

-
- “To overcome the limitations of the standard Round Robin arbiter—such as higher latency, sequential request checking, and poor scalability—we have used a Matrix Round Robin Arbiter, which provides faster parallel arbitration and better fairness.”

❖ **References:**

- 1) Toe, A., “Design and Verification of a Round-Robin Arbiter,” RIT Digital Institutional Repository, Rochester Institute of Technology.
- 2) M. Weber, “*Arbiters: Design Ideas and Coding Styles*,” IEEE.

Thank You!....